

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

6. Source Code

Source: Github

Github Repository Link:

https://github.com/ssvibitha/DeepLearningLab_2026-27/blob/main/Lab2_MLP/DL_Lab2.ipynb

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	rmsprop
Activation Function	tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	0.8906000000000001
Testing Accuracy	0.8825

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.865900	0.882500
Precision	0.877705	0.882713
Recall	0.865900	0.882500
F1-score	0.863692	0.882355
Training Time	113.793586 seconds	153.884540 seconds

9. Plots with Inference

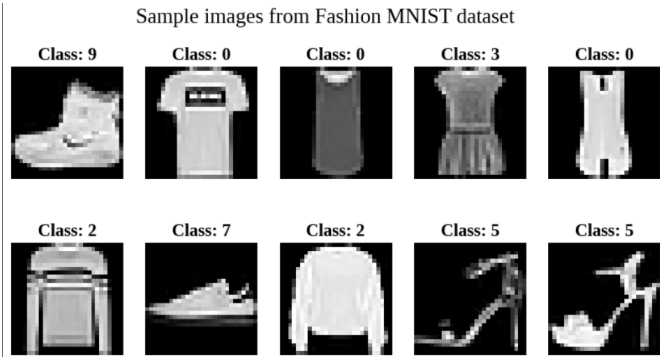


Figure 1: Sample Images

Sample Images Inference:

The sample images represent different fashion items with distinct shapes and patterns. These variations help the model learn and distinguish between different clothing classes.

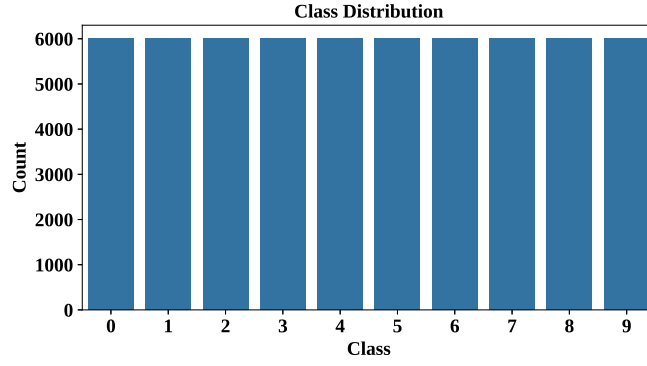


Figure 2: Class Distribution

Class Distribution Inference:

The plot shows that there exists 10 different fashion item classes. Each class has 6000 samples.

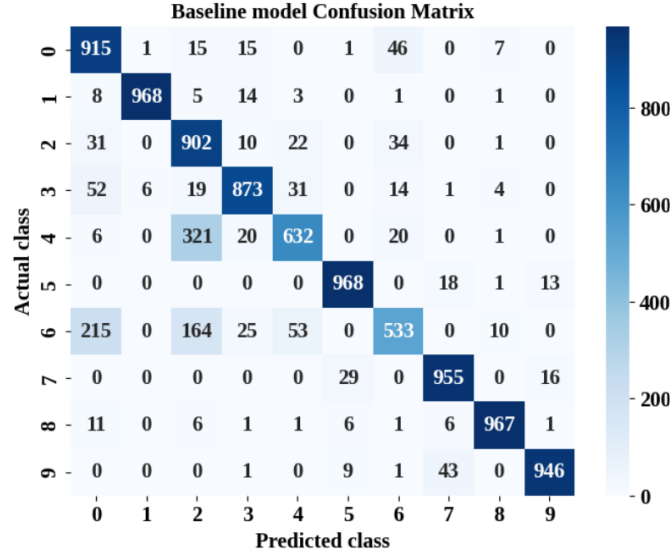


Figure 3: Baseline Model ConfusionMatrix

Baseline Model Confusion Matrix Inference:

The model correctly predicts most classes, performing best on classes 1, 5, 7, 8, and 9. It misclassifies class 4 as class 2 and class 6 as class 2 or class 0 often.

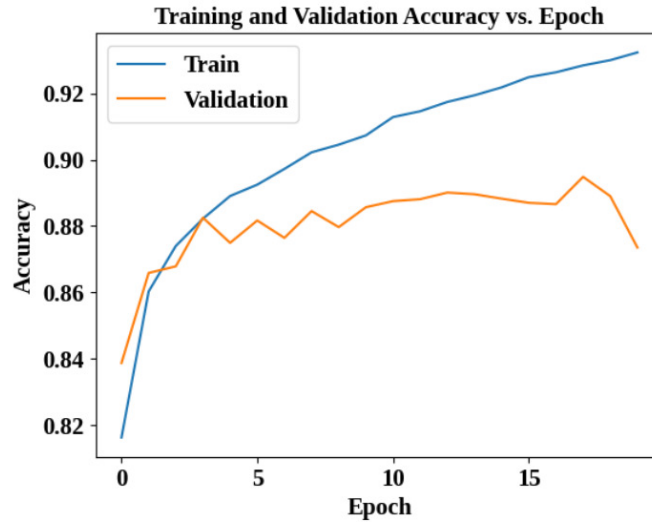


Figure 4: Training, Validation Accuracy Vs Epoch

Training, Validation Accuracy Vs Epoch Inference:

Training accuracy steadily increases throughout training, reaching over 93 percent showing the model is learning the training data well. Validation accuracy plateaus around 89 percent after epoch 5 and drops sharply at the end indicating the model stops improving on unseen data and begins to overfit.

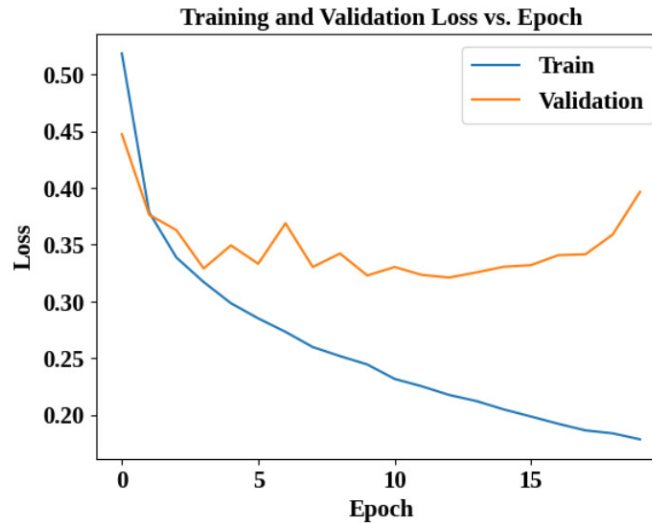


Figure 5: Training, Validation Loss Vs Epoch

Training, Validation Loss Vs Epoch Inference:

Training loss continuously decreases towards 0.18 showing consistent optimization on the training set. Validation loss decreases initially but hits a minimum around epoch 10 and 15 and begins rising sharply, clearly confirming overfitting in later epochs.

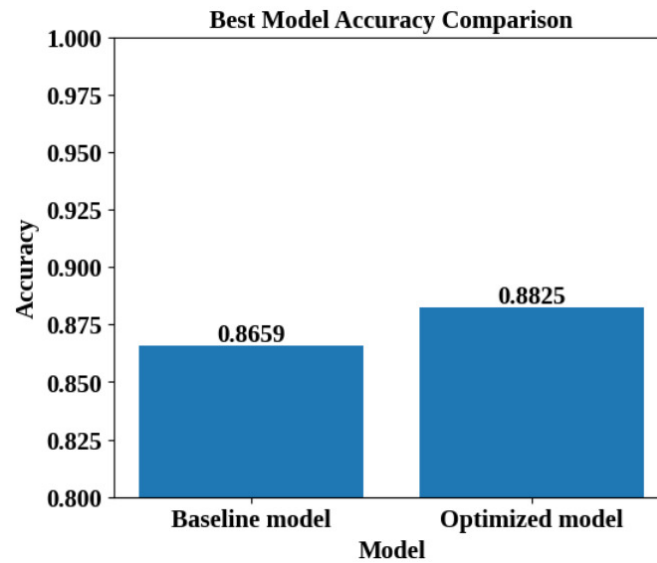


Figure 6: Best Model Accuracy Comparison

Best Model Accuracy Comparison Inference:

The optimized model achieved a higher accuracy of 88.25 percent compared to the baseline model's 86.59 percent. This indicates that hyperparameter tuning improved the model's classification performance.

10. Additional Task

Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.

1 Source Code

https://github.com/ssvibitha/DeepLearningLab_2026-27/blob/main/Lab2_MLP/AdditionalTask_2.ipynb

2 Plots and Inference

XOR Gate - Decision Boundaries During Training

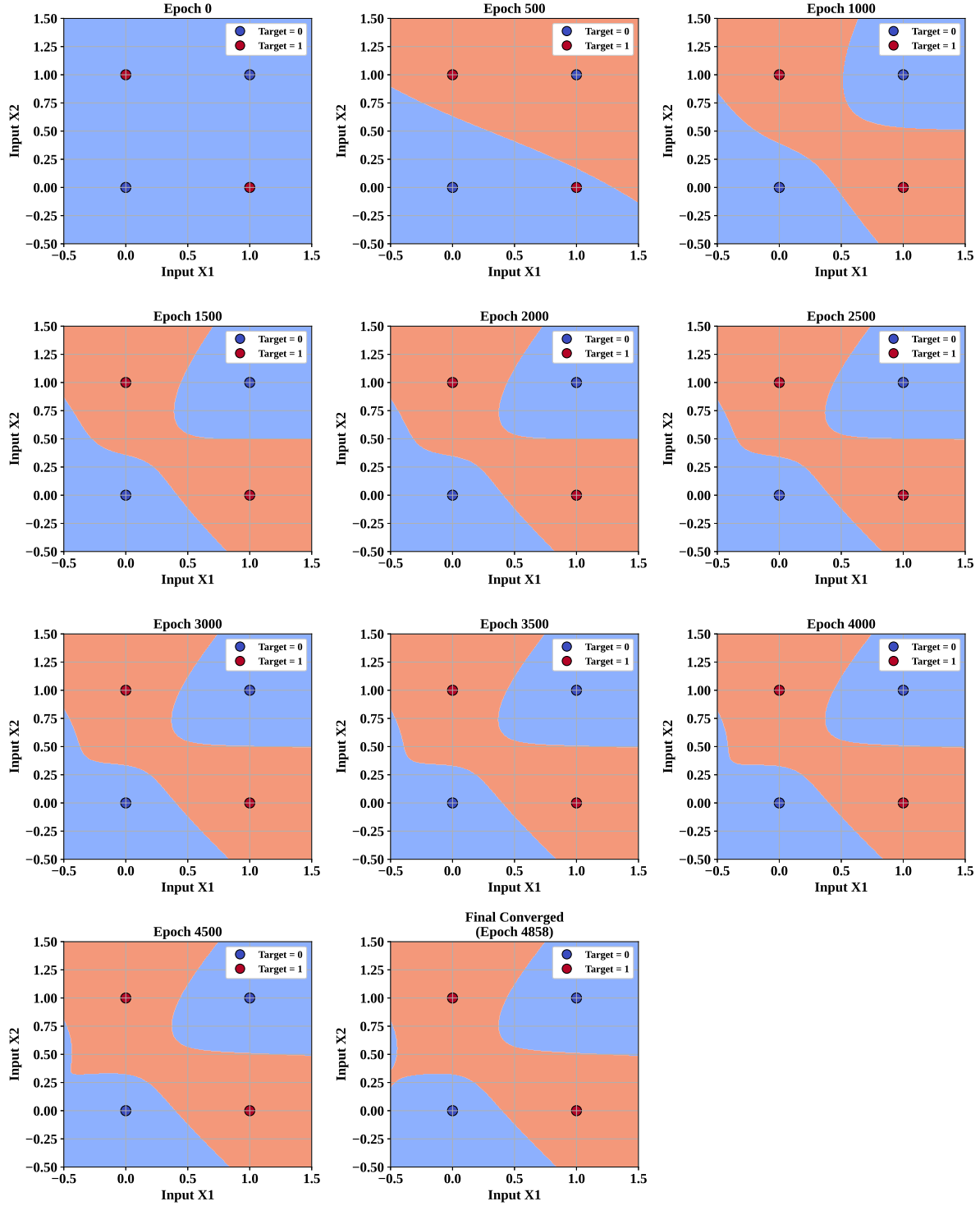


Figure 7: Multi-Layer Perceptron Training for XOR Gate

Multi-Layer Perceptron Training for XOR Gate Inference:

- The figure shows how the decision boundary changes during the training of a Multi-Layer Perceptron for XOR gate. As training progresses, the model learns from errors and adjusts its weights to create a better separation between the classes.
- **Epoch 0: Initial State**
 - At the beginning of training, the model starts with randomly initialized weights.
 - Since the model has not learned any patterns yet, it predicts the same class for the entire input space, resulting in a single decision region.
- **Epoch 500: Initial Boundary Formation**
 - A basic linear decision boundary starts to appear as the model begins learning from the data.
 - However, the boundary is not sufficient to classify the points correctly because the given pattern requires a non-linear separation.
- **Epochs 1000–2500: Learning Non-Linear Patterns**
 - As training continues, the hidden layers update their weights through backpropagation.
 - The decision boundary starts changing from a straight line into a curved shape, allowing the model to learn more complex relationships.
 - The model begins creating separate regions for correctly classifying the points:
 - * **Target = 1:** (0, 1) and (1, 0)
 - * **Target = 0:** (0, 0) and (1, 1)
- **Epochs 3000–4500: Refinement of Decision Regions**
 - The model continues adjusting its weights to improve classification accuracy.
 - The decision boundaries become more defined and gradually move to better separate the two classes.
 - The network becomes more confident in classifying each input point correctly.
- **Epoch 4858: Final Converged State**
 - After several training iterations, the model reaches a stable state where the decision boundaries correctly classify all the input points.
 - The points (0, 1) and (1, 0) are classified as **Target = 1**, while (0, 0) and (1, 1) are classified as **Target = 0**.
- **Training Significance**
 - The movement of the decision boundary across epochs shows how the neural network learns and improves through repeated weight updates.
 - The formation of curved boundaries highlights the ability of hidden layers to learn non-linear patterns, unlike a Single Layer Perceptron which can only create linear boundaries.
 - The final stable decision regions show that the network has successfully learned the classification pattern and achieved convergence.

11. Discussion

Answer the following:

1. Which optimization method was used?

Randomized Search with Cross-Validation is used to find the best hyperparameters.

2. Which hyperparameters were selected?

The selected hyperparameters are

- Hidden Layers: 3

- Hidden Neurons: 128
 - Learning Rate: 0.001
 - Batch Size: 32
 - Optimizer: RMSprop
 - Activation Function: tanh
 - Epochs: 30
 - Dropout: 0.2
3. **Did optimization improve performance?**
Yes, after optimization the testing accuracy improved from 86.59 percent to 88.25 percent, along with improved precision, recall, and F1-score.
 4. **Which hyperparameter had the greatest impact?**
The optimizer RMSprop and learning rate of 0.001 had the greatest impact helping improve the model's performance.
 5. **Compare Grid Search and Randomized Search.**
Grid Search tests every possible combination of hyperparameters. Hence it is more computationally expensive. Randomized Search tests a random subset of combinations, and finds good results with less computation.
 6. **Recommend the best MLP configuration.**
MLP configuration with 3 hidden layers, 128 neurons, tanh activation, RMSprop optimizer, learning rate 0.001, batch size 32, dropout 0.2, and 30 epochs is recommended as it achieved the highest testing accuracy of 88.25 percent.

12. Conclusion

The experiment successfully trained and evaluated an MLP classifier on the Fashion MNIST dataset and trained MLP on XOR gate. Hyperparameter tuning in training the Fashion MNIST dataset improved the model's accuracy and overall classification performance, resulting in a more effective classifier.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.